

CloudLab

Windows Installation Guide

Installing and running CloudLab on a personal Windows PC
via WSL2 (Ubuntu) + Docker Desktop

Generated: September 08, 2026

[WSL2 + Ubuntu](#)[Docker Desktop](#)[Postgres 16](#)[Redis](#)[Go](#)[Node.js 20+](#)

Why WSL2? CloudLab provisions real infrastructure using Linux-only mechanisms — real iptables rules for firewalls/NAT/VPC peering, a real dedicated PostgreSQL server per managed database, and a real Docker daemon. None of that exists natively on Windows, so CloudLab runs inside a lightweight Linux environment (WSL2) that Windows hosts directly — no separate physical machine or full virtual machine needed.

1 Before you start

This guide assumes a normal Windows 10 (version 2004+) or Windows 11 PC with an Intel or AMD processor (the common case — see the note on checking this in Part B). You'll go back and forth between two places throughout this guide:

TAG	MEANING
WINDOWS	Run this in a regular Windows PowerShell window, or by clicking things in a Windows app (Docker Desktop's settings, an installer, etc.)
UBUNTU	Run this inside the Ubuntu terminal window (opened from the Start menu after WSL2/Ubuntu is installed) — this is where CloudLab itself actually runs.

Every restart mentioned in this guide is a normal Windows restart or a lightweight WSL/Docker-engine restart — nothing here is destructive to your files, installed programs, or Windows itself. Where a step says "reboot," it means the same thing as installing a Windows update: close your other work, restart, and continue when it's back.

If you open this project in VS Code: VS Code's built-in terminal defaults to a plain Windows terminal (PowerShell or Command Prompt) — *not* the Ubuntu/WSL terminal this guide means by **UBUNTU** , even though it looks similar and sits in the same editor window. Running Linux-only commands there fails in confusing ways: `cp`, `grep`, and `unzip` come back as "not recognized," and `sudo` behaves completely differently (Windows 11's own built-in `sudo` doesn't accept `sudo -u postgres ...` the way Linux's does). Two ways to avoid this: either keep using the standalone Ubuntu terminal window (from the Start menu) for every command in this guide and only use VS Code to browse/edit files, or in VS Code's terminal panel click the small dropdown arrow next to the + button and choose an **Ubuntu (WSL)** profile if one is listed. A quick way to check which terminal you're actually in: run `pwd` — a path like `/home/yourname/...` means you're in Ubuntu/WSL (correct); a path like `C:\Users\...` means you're in a Windows terminal (wrong for this guide's commands).

2 Part A — Install WSL2 and Ubuntu

2.1 Enable WSL2 and install Ubuntu WINDOWS

Open **PowerShell as Administrator** (right-click the Start button → "Terminal (Admin)" or "Windows PowerShell (Admin)") and run:

```
wsl --install -d Ubuntu
```

This enables the underlying Windows virtualization features and begins installing Ubuntu. The very first time you run this, it will typically only get as far as enabling those features and then say it needs a restart — that's expected, not an error.

2.2 Reboot WINDOWS

Restart your PC normally (Start menu → Power → Restart, or run Restart-Computer from the same PowerShell window). This is required once, to activate WSL2's virtualization feature — it will not affect your files or other programs.

2.3 Finish installing Ubuntu WINDOWS

After the reboot, open PowerShell as Administrator again and run the same command a second time:

```
wsl --install -d Ubuntu
```

This time the virtualization features are active, so it will actually download and install Ubuntu (a few minutes). Confirm it succeeded:

```
wsl -l -v
```

You should see Ubuntu listed with VERSION = 2.

2.4 Create your Linux account UBUNTU

Ubuntu will launch automatically and ask you to create a **default Unix user account** — this is a Linux username/password, completely separate from your Windows login:

- **Username:** any lowercase name with no spaces (e.g. cloudlab, or whatever it suggests).
- **Password:** any password you choose — it does not need to match your Windows password. Remember it, since you'll need it for every `sudo` command below.

Note: when you type the password, nothing appears on screen at all — no dots, no cursor movement. That's normal Linux terminal behavior, not a bug. Type carefully and press Enter.

Once done, you'll land at a prompt like `yourname@DESKTOP-XXXX:~$` — you're now inside Ubuntu. Confirm `sudo` works:

```
sudo apt-get update
```

3 Part B — Install Docker Desktop

3.1 Check your CPU architecture WINDOWS

Almost all Windows PCs are **AMD64** (Intel or AMD processors) — ARM64 only applies to a small number of "Windows on ARM" devices (Surface Pro X, some Copilot+ laptops). To confirm, run in PowerShell:

```
systeminfo | findstr /B /C:"System Type"
```

x64-based PC → download **AMD64**. ARM64-based PC → download **ARM64**. If unsure, AMD64 is correct for the overwhelming majority of machines.

3.2 Download and install Docker Desktop WINDOWS

1. Download the installer for your architecture from docker.com/products/docker-desktop.
2. Double-click the downloaded `Docker Desktop Installer.exe`.
3. Leave **"Use WSL 2 instead of Hyper-V"** checked (default).
4. When asked **"Install for me only" vs "all users"**: choose **"Install for me only"** (the recommended default) unless multiple Windows logins on this PC need it.
5. When asked about **Windows containers**: say **No / leave unchecked**. CloudLab needs Linux containers (the default) — Windows containers are a different, unrelated feature for legacy Windows-only apps.
6. Click Install/OK and wait. The "Extracting manifest" step can genuinely take several minutes, especially with antivirus scanning each file — see the troubleshooting table in §7 if it seems frozen.
7. Reboot when prompted (a normal restart).

3.3 First launch WINDOWS

Docker Desktop should open automatically after the reboot (or launch it from the Start menu). Accept the service agreement. When asked to sign in, look for a smaller **"Skip for now"** / **"Continue without signing in"** option — a Docker account isn't needed for anything CloudLab does. Wait until the whale icon in the system tray stops animating.

3.4 Enable WSL integration WINDOWS

1. In Docker Desktop, click the gear icon (**Settings**) top right.
2. Go to **Resources → WSL Integration**.
3. Turn the toggle for **Ubuntu ON**.
4. Click **Apply & Restart** (this restarts Docker's engine only — a few seconds, not a full PC reboot).

3.5 Verify from Ubuntu UBUNTU

Open a fresh Ubuntu terminal (Start menu → "Ubuntu") and run:

```
docker run hello-world
```

Expected output:

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
...
```

If you instead see permission denied while trying to connect to the docker API at `unix:///var/run/docker.sock`: this is a common first-time hiccup, not a real problem — see §7's troubleshooting table for the exact fix (a WSL restart almost always resolves it).

4 Part C — Install the remaining prerequisites

All commands in this section run inside **UBUNTU**.

4.1 Node.js 20+

Ubuntu's built-in nodejs package is usually too old for this project, so install it from NodeSource's official repository instead:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Confirm: `node --version` should print `v20.x.x` or higher.

4.2 PostgreSQL 16 — add the official repository first

Ubuntu's own default package repository doesn't always include a package literally named `postgresql-16` (it depends on which Ubuntu release you're on) — running `apt-get install postgresql-16` directly can fail with `E: Unable to locate package postgresql-16`. Avoid this by adding PostgreSQL's own official repository first, which always has version 16 available:

```
sudo apt-get install -y curl ca-certificates
sudo install -d /usr/share/postgresql-common/pgdg
sudo curl -o /usr/share/postgresql-common/pgdg/apt.postgresql.org.asc --fail https://www.postgresql.org/media/keys/ACCC4CF8.asc
```

Now add the repository line itself. Earlier revisions of this guide tried to auto-detect your Ubuntu codename (via `lsb_release -cs`, then via `/etc/os-release`) — both can come back with an interim/very-new codename (e.g. `resolute`) that PostgreSQL's official repository doesn't recognize yet, which `apt` then reports as a confusing Malformed entry ... (Suite) parsing error. The reliable fix is to skip auto-detection and hardcode a known-good Ubuntu **LTS** codename — `noble` (Ubuntu 24.04) — which PGDG is guaranteed to support regardless of which actual Ubuntu release your WSL image reports. This also uses `sudo tee` instead of `sudo sh -c "echo ... > file"`, which is a more robust way to write a root-owned file:

Important: the repository line below is long, and a few PDF viewers insert an actual line break into copied text wherever a line visually wraps on the page. If you copy-paste it as one block and it lands in your file as two lines, PostgreSQL's repository will be rejected as malformed. To avoid this entirely, build the line from short pieces instead of copying one long line:

```
sudo rm -f /etc/apt/sources.list.d/pgdg.list

K=/usr/share/postgresql-common/pgdg/apt.postgresql.org.asc
R=https://apt.postgresql.org/pub/repos/apt
L="deb [signed-by=$K] $R noble-pgdg main"

echo "$L" | sudo tee /etc/apt/sources.list.d/pgdg.list

cat -A /etc/apt/sources.list.d/pgdg.list
```

Every line in that block is short enough that it can't get wrapped and split by a PDF viewer's copy behavior — the full repository line is only assembled at the very end, inside Ubuntu's own shell. The final `cat -A` command prints hidden characters: you should see exactly **one** line, starting with `deb` and ending in `main$` (a single `$` = end of line, with no stray `^M` and no second line). If you see the repository text split across two lines ending in `$` instead of one, that confirms the copy-paste line-break problem — re-run the block above and make sure each of the four short lines (`K=...`, `R=...`, `L=...`, `echo...`) is pasted as its own line, not merged or split further. It's safe to use `noble` here even if your WSL Ubuntu install isn't literally 24.04 — PGDG's package contents for a given PostgreSQL major version are the same across recent Ubuntu releases, so pointing at the `noble-pgdg` suite works whether you're on 22.04, 24.04, or a newer interim release. Then:

```
sudo apt-get update
```

If you still see Malformed entry ... in list file ... (Suite) **or** (Component) **after this:** the `cat -A` output has more than one line, or a stray `^M` in it. Remove the file (`sudo rm -f /etc/apt/sources.list.d/pgdg.list`) and redo the four-line block above exactly, pasting or typing each short line separately rather than the whole thing as one long line.

If you already hit the earlier E: Unable to locate package postgresql-16 error: once `apt-get update` above runs cleanly (no errors), move on to §4.3.

If `apt-get install` in §4.3 instead fails with `Depends: libicu74 ... but it is not installable (or a similar unmet dependency on libicu74/libxml2)`: this means your WSL Ubuntu image is itself a release newer than 24.04 (e.g. codename `resolute` or later), and its own base libraries (ICU, libxml2, etc.) have already moved past the versions PostgreSQL's `noble`-targeted package was built against — an old repository pointed at a brand-new OS. There's no combination of flags that fixes this; the PGDG repository simply doesn't have a build for your OS yet. Drop it and use Ubuntu's own built-in PostgreSQL package instead, which will already match your OS's libraries — CloudLab works fine on any modern Postgres version, it doesn't require exactly 16.x:

```
sudo rm -f /etc/apt/sources.list.d/pgdg.list
sudo apt-get update
```

Then use the fallback install command in §4.3 (Option B) instead of Option A.

4.3 Install PostgreSQL, Redis, Go, and networking tools

Option A — if the §4.2 PGDG repository is working (no errors when you ran `apt-get update`, and no `libc74/libxml2` conflict below):

```
sudo apt-get install -y postgresql-16 redis-server golang busybox-static iptables
```

Option B — fallback, if you hit the `libc74/libxml2` conflict above and removed the PGDG repository: install Ubuntu's own default PostgreSQL package instead of asking for version 16 specifically — `apt` will pick whatever version matches your OS's own libraries:

```
sudo apt-get install -y postgresql postgresql-contrib redis-server golang busybox-static iptables
```

Only run **one** of Option A or Option B, not both. If Option A's combined command fails with the `libc74` error, that command installs *none* of the five packages (`apt` aborts the whole transaction) — just run Option B in full, it covers all five packages again.

Confirm each installed:

```
psql --version
redis-cli --version
go version
```

Each should print a real version number — with Option B, `psql --version` will show whatever version Ubuntu shipped (17, 18, or newer), which is fine.

5 Part D — Get the CloudLab source and configure it

All commands in this section run inside  .

5.1 Unzip the source into your Linux home directory

Put it under your Ubuntu home directory (~), not the Windows C: drive (/mnt/c/...) — file access across that boundary is much slower and can cause odd Docker permission issues.

```
cd ~
unzip /mnt/c/Users/<your-windows-username>/Downloads/CloudLab_Source_Complete_v3.zip -d cloudlab
cd cloudlab
npm install
```

5.2 Configure environment variables

Copy the template config file, then generate a random encryption key and write it straight into .env — no text editor needed:

```
cp .env.example .env
KEY=$(node -e "console.log(require('crypto').randomBytes(32).toString('base64'))")
sed -i "s|^SECRETS_MASTER_KEY=.*|SECRETS_MASTER_KEY=$KEY|" .env
grep SECRETS_MASTER_KEY .env
```

The SECRETS_MASTER_KEY is the master key CloudLab uses to encrypt sensitive data at rest inside the app (the "secrets" resource type, service-account private keys, etc.) — it just needs to be random and stay inside this .env file, nothing to remember or look up. The final grep command should print a line like SECRETS_MASTER_KEY=K8f3JHb2mZpQr...== confirming it was written; if the value after = is empty, re-run the three commands above.

Prefer to see/edit the file yourself? nano .env opens it in a simple in-terminal editor — arrow keys move the cursor, typing or pasting inserts text, Ctrl+O then Enter saves, Ctrl+X exits. The commands above do the same edit automatically, without opening it.

5.3 Set up Postgres for CloudLab

```
sudo -u postgres psql -c "ALTER USER postgres WITH PASSWORD 'cloudlab';"
sudo -u postgres createdb cloudlab
npm run db:migrate
```

Already have pgAdmin on Windows? WSL2 forwards localhost to Windows automatically, so you can register a server in pgAdmin pointing at host localhost, port 5432, user postgres, password cloudlab — you'll see the cloudlab database appear there once migrations run, handy for browsing tables visually.

The app also provisions **real, separate Postgres servers on demand** (one per managed-database or BigQuery-dataset resource you create in the app), which needs passwordless sudo access to the postgres OS account:

```
echo "${whoami} ALL=(postgres) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/cloudlab-worker
```

5.4 Build the local container images

Nothing is pulled from a registry here — every image is compiled and built from the included source:

```
bash infra/guest-image/build.sh
bash infra/dns-image/build.sh
bash infra/lb-image/build.sh
bash infra/registry-image/build.sh
bash infra/bucket-image/build.sh
```

6 Part E — Run and verify CloudLab

All commands in this section run inside , from the `~/cloudlab` directory.

6.1 Start the two long-running processes

Open two separate Ubuntu terminal windows.

```
# terminal 1
npm run dev:api
```

```
# terminal 2 -- must run with root privileges: it calls real iptables/sysctl
# commands directly for firewall rules, NAT gateways, and VPC peering
sudo -E npm run dev:worker
```

6.2 Open the console

In your normal Windows browser, go to:

```
http://localhost:4000
```

WSL2 forwards this port to Windows automatically — no extra configuration needed. Register your first account and you're in.

6.3 Run the automated checks (optional but recommended)

```
npm test          # 20 checks against the API layer
npm run e2e       # 31 checks against the full running stack
```

Both should finish reporting all checks passed — a good sign your install is fully healthy end to end.

7 Troubleshooting reference

Real issues that came up while working through this exact install, and their fixes:

SYMPTOM	FIX
<code>wsl -l -v</code> says "no installed distributions" right after <code>wsl --install</code>	Expected the first time — the command only enabled Windows features so far. Reboot, then run <code>wsl --install -d Ubuntu</code> again.
Docker Desktop installer stuck at "Extracting manifest"	Check Task Manager for the installer's CPU/Disk activity — if non-zero, it's just slow (antivirus scanning), wait 5–10 min. If truly 0% for a long time: End task, temporarily disable antivirus real-time protection, re-download the installer, run it as Administrator, retry.
Docker Desktop asks to sign in on first launch	Look for a smaller "Skip for now" link below the main sign-in button. Not required for CloudLab either way.
<code>docker run hello-world</code> → "permission denied ... docker.sock"	Close all Ubuntu windows → run <code>wsl --shutdown</code> in PowerShell → confirm Docker Desktop's WSL Integration toggle for Ubuntu is still on (toggle off/on and Apply & Restart if unsure) → reopen Ubuntu → retry. If it persists: <code>sudo usermod -aG docker \$USER</code> , then repeat the <code>wsl --shutdown</code> + reopen step.
<code>node --version</code> shows below v20 after <code>apt-get install nodejs</code>	Use the NodeSource setup script in §4.1 instead of the plain <code>apt-get install nodejs</code> — Ubuntu's default repo version is usually too old.
E: Unable to locate package <code>postgresql-16</code>	Ubuntu's default repo doesn't always carry a <code>postgresql-16</code> package by that exact name. Add PostgreSQL's official repository first (§4.2), then re-run the full install command in §4.3 — not just for Postgres, since the earlier failed command installed none of the five packages.
E: Malformed entry 1 in list file <code>.../pgdg.list</code> (Suite) after adding the PostgreSQL repository	Auto-detecting your Ubuntu codename (via <code>lsb_release -cs</code> or <code>/etc/os-release</code>) can return a very new/interim codename (e.g. <code>resolute</code>) that PostgreSQL's repository doesn't recognize yet. Run <code>sudo rm -f /etc/apt/sources.list.d/pgdg.list</code> , then use the hardcoded-noble method in §4.2 instead — it points at a known-good LTS codename that PGDG always supports, regardless of your actual Ubuntu release.
E: Malformed entry 1 in list file <code>.../pgdg.list</code> (Component) — and <code>cat -A</code> shows the repository text split across <i>two</i> lines ending in <code>\$</code>	The repository line is long enough that it visually wraps on the PDF page, and some PDF viewers insert a real line break into copied text at that wrap point — so pasting it as one block silently splits it into two lines in the file (e.g. <code>...noble-pgdg</code> on one line, <code>main</code> alone on the next). Run <code>sudo rm -f /etc/apt/sources.list.d/pgdg.list</code> , then use the short-variable version of the command in §4.2 (<code>K=... / R=... / L=... / echo "\$L" sudo tee ...</code>) — each line is short enough that it can't be wrapped and split by a PDF viewer.
<code>postgresql-16</code> : Depends: <code>libc6</code> ... but it is not installable (or similar, for <code>libxml2</code>) when running §4.3's install command	Your WSL Ubuntu image is a release newer than 24.04 (e.g. codename <code>resolute</code> or later) whose own base libraries have already moved past what PostgreSQL's noble-targeted package needs — a real, unfixable ABI mismatch, not a typo or a paste error. Run <code>sudo rm -f /etc/apt/sources.list.d/pgdg.list && sudo apt-get update</code> , then use §4.3's Option B (<code>sudo apt-get install -y postgresql postgresql-contrib redis-server golang busybox-static iptables</code>) instead of Option A — it installs whichever PostgreSQL version Ubuntu itself ships, which CloudLab works fine with.
'cp' is not recognized..., 'grep' is not recognized..., or <code>sudo -u postgres ...</code> → error: unexpected argument '-u' found	You're typing Ubuntu-only commands into a Windows terminal (Command Prompt or PowerShell) instead of the Ubuntu/WSL terminal — very easy to do by accident if the project is open in VS Code, since its default integrated terminal is a Windows shell even though it looks similar. Windows has no <code>cp/grep</code> , and Windows 11's own built-in <code>sudo</code> command takes different flags than Linux's. Switch to the actual Ubuntu terminal window (see the callout in §1), <code>cd ~/cloudlab</code> , and re-run the command there. Run <code>pwd</code> if you're ever unsure which terminal you're in.
Worker process errors touching <code>iptables/sysctl</code>	The worker must be started with <code>sudo -E npm run dev:worker</code> , not a plain <code>npm run dev:worker</code> — it needs root privileges for real firewall/NAT/peering rules.
Postgres-backed features (managed databases, BigQuery datasets) fail to provision	Confirm the sudoers line from §5.3 was added correctly: <code>cat /etc/sudoers.d/cloudlab-worker</code> should show your username with NOPASSWD access to the postgres account.

8 Quick-reference checklist

STEP	
☐	WSL2 + Ubuntu installed, Linux user account created
☐	Docker Desktop installed (correct CPU architecture, Linux containers, WSL integration enabled)
☐	<code>docker run hello-world</code> succeeds from Ubuntu
☐	Node.js 20+, PostgreSQL (16 via PGDG, or your OS's default version), Redis, Go, busybox-static, iptables installed
☐	CloudLab source unzipped under <code>~/cloudlab</code> (not <code>/mnt/c/...</code>), <code>npm install</code> run
☐	<code>.env</code> configured with a generated <code>SECRETS_MASTER_KEY</code>
☐	Postgres role password set, cloudlab database created, migrations run, sudoers NOPASSWD entry added
☐	All 5 local container images built (<code>infra/*/build.sh</code>)
☐	<code>npm run dev:api</code> and <code>sudo -E npm run dev:worker</code> both running
☐	<code>http://localhost:4000</code> loads in your Windows browser, account registered
☐	<code>npm test</code> and <code>npm run e2e</code> both pass

This guide reflects a real, working WSL2 + Docker Desktop installation path for a personal Windows machine with an Intel/AMD (AMD64) processor. Every command and troubleshooting fix here was verified against the actual CloudLab source and a real install session.